Speak with Krishna — Project Architecture

Document for: Tech Lead / engineering review
Product name: bhagavad-gita-chat (Speak with Krishna)
Stack: Next.js 15 · React 19 · TypeScript · Groq · Jina · ElevenLabs (optional)
Repo path: `d:\xvox\bhagavad_gita`

---

1. Product summary

A web app where a person describes a personal situation and receives brief guidance in Krishna's voice, grounded only in real Bhagavad Gita verses. The same app also lets users read all 18 chapters.

Feature	Route / area	What it does
Chat	`/`	Situation → guidance + verified verse cards (streamed)
Chapter reader	`/gita`, `/gita/[chapter]`	Browse 18 chapters / 701 verses
Verse of the Day	Home screen	One daily verse (IST day rollover)
My Gita	Chat UI	Save / unsave verses in the browser
Share	Chat / reader	Share or download a verse image card
Voice in / out	Mic + speak	Browser speech, or ElevenLabs when keys are set
Multi-language reply	Chat pipeline	Reply language detected from the user's message

Accuracy rule (non-negotiable): the model writes guidance only. The **server** chooses and attaches verse text from local files. Sanskrit is never taken from the model.

---

2. Architecture (current)

```
Browser (React UI)
   |
   |-- POST /api/chat     --> Chat pipeline (Node)
   |-- GET  /api/tts      --> Text-to-speech
   |-- POST /api/stt      --> Speech-to-text
   |-- GET  /api/voice    --> Voice feature flags (tts/stt enabled?)
   |
Chat pipeline
   |-- 1. Understand situation     --> Groq LLM
   |-- 2. Embed query + shortlist  --> Jina + local vectors
   |-- 3. Select citations         --> Groq (shortlist only)
   |-- 4. Verify citations         --> Local corpus lookup
   |-- 5. Write guidance           --> Groq
   |-- 6. Attach stored scripture  --> verse.json + CSV + enrichment
   |
Local data (source of truth)
   verse.json | Bhagwad_Gita.csv | verse-enriched.json | verse-embeddings.json
```

Deployment shape: one Next.js app (UI + API). No separate backend service. No Postgres yet; sessions are in-memory.

---

3. Chat pipeline (step by step)

1. **Understand** — Groq returns JSON: intent, reply language, emotions, themes, crisis flag.
2. **Search** — Build ~18 candidate verses using:
   - cosine similarity on Jina embeddings (`verse-embeddings.json`)
   - theme / keyword overlap (`themes.ts` + enrichment)
   - teaching / "not for" filters from enrichment
3. **Choose** — Groq may cite only IDs from that shortlist.
4. **Verify** — Server looks up each ID in the corpus; invalid IDs dropped; one retry allowed.
5. **Write** — Groq writes acknowledge / connection / step text. No Sanskrit, no chapter.verse numbers in guidance.
6. **Attach** — Server copies Sanskrit, transliteration, word meanings, English/Hindi, summary, themes from files.
7. **Stream** — Optional NDJSON: stage → verses → sentence deltas → final `done` payload.

Non-problem intents (greeting, about-me, unclear, off-topic) get a conversation reply with **no verse**.

---

4. Technology used (and why)

Technology	Version / default	Purpose in this project
Next.js	^15.5.4 (App Router)	Pages, SSR/ISR reader, API routes in one deployable unit
React	^19.1.1	Chat screen, reader UI, client hooks (speech, saved verses)
TypeScript	^5.9.2	Typed corpus, pipeline, API contracts
Node.js runtime	`runtime = 'nodejs'` on APIs	Read local files; call Groq / Jina / ElevenLabs; `maxDuration` 60s
openai npm SDK	^5.23.2	Client library pointed at Groq's OpenAI-compatible base URL
Groq	default model `openai/gpt-oss-120b`	Situation extract, verse select, write / converse (JSON mode)
Jina AI	`jina-embeddings-v4`, 1024 dims	Query embedding + precomputed verse vectors for retrieval
ElevenLabs	TTS `eleven_v4`, STT `scribe_v2`	Optional natural voice; browser Web Speech is the fallback
tsx + node:test	^4.20.5	Unit tests and offline scripts

Intentionally not used yet

Item	Reason

Postgres / pgvector	File-based corpus is enough for current phase
Full 701-verse prompt every request	Exceeds Groq on-demand TPM (~8k); shortlist keeps prompts small
Model-generated Sanskrit	Would break scripture accuracy

---

5. Current data inventory

File	Approx size	Records	Role
`verse.json`	~644 KB	**701** verses	Immutable Sanskrit, transliteration, word meanings
`Bhagwad_Gita.csv`	~1.5 MB	**701** rows	English + Hindi prose meanings
`verse-enriched.json`	~315 KB	**701**	Themes, summary, situations, not-for, teaching flag
`verse-embeddings.json`	~3.7 MB	**701** × 1024-d	Precomputed Jina vectors (`jina-embeddings-v4`)
`src/lib/eval/golden-set.ts`	—	**58** cases	Citation accuracy eval situations

`verse.json` is never edited in place. Bad Latin fields on a few verses (e.g. 1.5, 1.6, 10.33, 11.19) are quarantined in code.

Offline scripts:

Command	Script	Output
`npm run enrich`	`scripts/enrich-verses.ts`	`verse-enriched.json`
`npm run embed`	`scripts/embed-verses.ts`	`verse-embeddings.json`
`npm run eval`	`scripts/run-eval.ts`	Golden-set citation check

---

6. Application map

Pages

Path	File	Purpose
`/`	`src/app/page.tsx` + `chat-screen.tsx`	Main chat + daily verse
`/gita`	`src/app/gita/page.tsx`	Chapter index
`/gita/[chapter]`	`src/app/gita/[chapter]/page.tsx`	Chapter verses

API routes

Endpoint	Method	Purpose

`/api/chat`	POST	Chat body `{ sessionId, message }` or `{ sessionId, reset: true }`; `stream: true` for NDJSON
`/api/tts`	GET	Speak a prior assistant reply
`/api/stt`	POST	Transcribe mic audio
`/api/voice`	GET	Reports whether ElevenLabs TTS/STT are configured

Limits: message ≤ **2000** chars; in-memory session ~**8** turns; chat `maxDuration` **60** s.

Core libraries

Module	Responsibility
`src/lib/pipeline.ts`	Orchestrates understand → search → choose → verify → write
`src/lib/llm.ts`	Groq calls (situation / select / write / converse / stream)
`src/lib/retrieve.ts`	Embedding + theme shortlist (~18 verses)
`src/lib/verify.ts`	Parse JSON; validate citations; leak / settle logic
`src/lib/corpus.ts`	Load verses; quarantine; display shape
`src/lib/embed.ts`	Jina embed API + load `verse-embeddings.json`
`src/lib/themes.ts`	Theme IDs and keyword maps
`src/lib/enrichment.ts`	Load enriched verse metadata
`src/lib/gita-csv.ts`	CSV English/Hindi + chapter yoga titles
`src/lib/safety.ts`	Crisis detection + helpline block
`src/lib/session.ts`	In-memory conversation store
`src/lib/rate-limit.ts`	Simple request limiting
`src/lib/stream-reply.ts`	Sentence streaming gate for UI
`src/lib/speech.ts` / `elevenlabs.ts`	Speakable text + voice provider
`src/lib/language.ts`	Language codes / display names
`src/lib/daily.ts`	Verse of the Day selection
`src/lib/chapters.ts`	Chapter metadata for reader

---

7. Accuracy and safety controls

- Shortlist allow-list: model cites only retrieved refs; server re-checks against corpus.
- Scripture copy: UI verse text comes from files, not from the model payload.
- Field quarantine for known bad transliteration / word-meaning rows.
- Guidance leak checks (verse numbers / Devanagari) → retry or fixed lead-in.
- Repeat-citation guard when the user raises a **new** situation.

- Crisis wording → fixed safety message + helplines (no clinical invention).
- Secrets only in `.env.local`; errors redact key patterns (Groq / Jina / ElevenLabs).

---

8. Environment variables (names only)

Variable	Service
`GROQ_API_KEY`, `GROQ_MODEL`	Chat LLM (default model `openai/gpt-oss-120b`)
`JINA_API_KEY`, `JINA_EMBED_MODEL`	Embeddings (default `jina-embeddings-v4`)
`ELEVENLABS_API_KEY`, `ELEVENLABS_VOICE_ID`	Voice
`ELEVENLABS_TTS_MODEL`, `ELEVENLABS_STT_MODEL`	Voice models (defaults `eleven_v4`, `scribe_v2`)

Never commit `.env.local`.

---

9. How to run

```
npm install
npm run dev      # http://localhost:3000
npm test         # unit tests (corpus, verify, retrieve, safety, speech, language, rate-limit, daily, stream)
npm run enrich   # refresh verse-enriched.json
npm run embed    # refresh verse-embeddings.json
npm run eval     # run 58-case golden citation eval
```

---

10. Status vs next work

In place now	Typical later work
Chat + streaming + multi-language replies	Durable Postgres sessions
Embedding retrieval + theme enrichment	Stronger production rate limits / observability
Citation verification + golden eval (58 cases)	Expand eval set; host on Vercel/similar
Chapter reader, save, share, daily verse	—
Optional ElevenLabs voice	—

---

11. One-line summary

One Next.js app: Groq for language, Jina for verse search, local Gita files as scripture source of truth, and server-side citation verification so the website never shows invented verses.